

UNIVERSIDAD DEL VALLE DE GUATEMALA

CC2017 – Modelación y Simulación ù Sección 30

OUTBREAK

STOCHASTIC SURVIVAL LAB

Simulación, validación y comparación de métodos de generación



Pablo Daniel Barillas Moreno ù Jorge Palacios
Andrés Rafael Chivalán ù Javier Andrés Chen

Proyecto de curso ù Agosto de 2026

Resumen

Este trabajo modela un subsistema real de un videojuego de supervivencia: la aparición, desplazamiento y presión concurrente de enemigos durante una misión. El modelo principal es un proceso de Poisson homogéneo cuyos interarribos exponenciales se generan por transformada inversa. Como contraste se implementa un proceso de renovación con interarribos lognormales; las normales auxiliares se obtienen mediante Marsaglia Polar. Ambos modelos comparten tiempo medio entre llegadas, pero difieren en variabilidad y memoria.

La versión auditada valida la cadena completa: primero la fuente uniforme que alimenta a ambos métodos, después las transformaciones y por último el conteo del calendario, con 18 contrastes que no se solapan entre sí y corrección por multiplicidad; comparación pareada con McNemar, Wilcoxon y bootstrap; intervalos de Wilson; benchmark de generación; corrección del límite temporal; figuras reproducibles y una escena tridimensional. En el escenario base, Poisson alcanzó 96.5 % de supervivencia y Polar-lognormal 100.0 %. El efecto Poisson menos Polar fue -3.5 puntos porcentuales, con IC bootstrap [-5.5, -1.8] y $p = 0.000122$. Los datos completos se distribuyen como CSV y pueden regenerarse desde el código.

Índice

1 Descripción del sistema	3
1.1 Sistema real modelado	3
1.2 Alcance	3
2 Objetivos	3
2.1 Objetivo general	3
2.2 Objetivos específicos	3
3 Fuentes de aleatoriedad	4
4 Fundamento matemático	4
4.1 Proceso de Poisson homogéneo	4
4.2 Marsaglia Polar y transformación lognormal	4
4.3 Conteo de renovación	5
4.4 Acumulación	5
5 Método de simulación	5
5.1 Algoritmo por paso	5
5.2 Parámetros base	6
6 Diseño experimental	6
6.1 Validación formal	6
6.2 Monte Carlo y Wilson	7
6.3 Inferencia pareada	7
6.4 Criterios de elección	7

7 Resultados reproducibles	7
7.1 Calidad de generadores	8
7.2 Control negativo	9
7.3 Costo de generación	10
7.4 Supervivencia y congestión	10
7.5 Sensibilidad a la tasa	11
7.6 Trayectoria individual	12
8 Análisis y elección	12
9 Visualización y arquitectura	12
10 Verificación	13
11 Reproducibilidad	14
12 Supuestos y limitaciones	14
13 Conclusiones	15
14 Trabajo futuro	15

1 Descripción del sistema

1.1 Sistema real modelado

La narrativa utiliza un apocalipsis zombi, pero el objeto técnico es el **subsistema de aparición y balance de enemigos de un videojuego**. Este tipo de sistema real de software decide cuándo aparece una entidad, qué atributos recibe y cómo la presión acumulada afecta la dificultad. Reemplazar apariciones periódicas por variables aleatorias evita patrones memorizables y permite cuantificar el balance por simulación.

El protagonista permanece en el centro de una arena. Los infectados nacen cerca del perímetro, avanzan radialmente y agregan su DPS al daño entrante cuando alcanzan el radio de contacto. El protagonista ataca un objetivo a la vez. La misión termina al alcanzar T con HP positivo o al agotar la vida.

Pregunta de investigación ¿Cómo cambian congestión, supervivencia y costo computacional cuando se conserva la tasa media, pero se modifica el método y la distribución de los interarribos?

1.2 Alcance

El motor procesa un calendario y actualiza combate con paso fijo. No pretende ser un motor formal de eventos discretos, formalismo que las instrucciones del curso declaran innecesario en esta etapa. Ruinas, vehículos y barricadas aportan contexto visual, pero no alteran trayectorias. Las salidas son supervivencia, tiempo resistido, llegadas, eliminaciones y concurrencia máxima.

2 Objetivos

2.1 Objetivo general

Diseñar, implementar y analizar una simulación reproducible del subsistema de aparición de enemigos, utilizando variables aleatorias continuas para estimar la probabilidad de completar una misión y estudiar el efecto de la estructura temporal sobre la congestión.

2.2 Objetivos específicos

1. Generar interarribos exponenciales mediante transformada inversa.
2. Implementar Marsaglia Polar y convertir sus normales en tiempos lognormales positivos.
3. Verificar distribución, momentos, independencia, dispersión y aceptación de los generadores.
4. Comparar modelos bajo la misma media mediante diseño pareado.
5. Estimar probabilidades e incertidumbre con Monte Carlo, Wilson y bootstrap.
6. Evaluar costo, interpretabilidad y comportamiento en tasas extremas.
7. Comunicar resultados mediante gráficos, escena 3D, presentación e informe reproducible.

3 Fuentes de aleatoriedad

La Tabla 1 documenta todas las fuentes y no solamente los interarribos.

Cuadro 1: Fuentes aleatorias, parametrización y justificación.

Fuente	Distribución	Justificación
Interarribo Poisson	$\Delta \sim \text{Exp}(\lambda)$	Tasa constante, independencia y falta de memoria.
Interarribo alternativo	Lognormal con $E[\Delta] = 1/\lambda$ y CV configurable	Tiempo positivo y regularidad controlable.
Normal auxiliar	$Z \sim N(0, 1)$ mediante Marsaglia Polar	Método visto en el curso; no se delega a una normal de biblioteca.
Ángulo	$U(0, 2\pi)$	Simetría alrededor de la arena.
Radio inicial	$U(0.94R, 1.03R)$	Variación espacial acotada.
Velocidad	$U(0.88v, 1.12v)$	Heterogeneidad sin valores extremos.
HP y DPS	$U(0.90b, 1.12b)$	Diferencias individuales conservando el balance.
Semillas	Enteros derivados de una semilla maestra	Reproducibilidad y emparejamiento de escenarios.

Llegadas y atributos se construyen con flujos separados mediante `SeedSequence.spawn`. Así, cambiar el algoritmo temporal no altera por consumo accidental el HP, velocidad o DPS del enemigo de igual índice.

4 Fundamento matemático

4.1 Proceso de Poisson homogéneo

Sea $N(t)$ el número de apariciones hasta t . Para $\lambda > 0$:

$$N(t) \sim \text{Poisson}(\lambda t), \quad P\{N(t) = k\} = e^{-\lambda t} \frac{(\lambda t)^k}{k!}. \quad (1)$$

$$\mathbb{E}[N(t)] = \text{Var}[N(t)] = \lambda t. \quad (2)$$

Los interarribos son exponenciales independientes y se obtienen por inversa:

$$U_i \sim U(0, 1), \quad \Delta_i = -\frac{\ln(U_i)}{\lambda}. \quad (3)$$

Usar $-\ln(U)$ o $-\ln(1 - U)$ es equivalente para una uniforme continua. Este procedimiento sigue la generación no uniforme descrita por Devroye [2].

4.2 Marsaglia Polar y transformación lognormal

Se proponen $V_1, V_2 \sim U(-1, 1)$ y $S = V_1^2 + V_2^2$. Si $0 < S < 1$:

$$Z_1 = V_1 \sqrt{\frac{-2 \ln S}{S}}, \quad Z_2 = V_2 \sqrt{\frac{-2 \ln S}{S}}. \quad (4)$$

Ambos siguen $N(0, 1)$ [3]. Como una normal puede ser negativa, se aplica:

$$\Delta_i = e^{\mu + \sigma Z_i}, \quad \sigma^2 = \ln(1 + c^2), \quad \mu = \ln(1/\lambda) - \frac{\sigma^2}{2}. \quad (5)$$

Así, $\mathbb{E}[\Delta] = 1/\lambda$ y c controla el CV.

4.3 Conteo de renovación

Igualar $\mathbb{E}[\Delta]$ iguala la tasa de largo plazo, pero en una renovación lognormal no implica $\mathbb{E}[N(t)] = \lambda t$ exactamente para todo horizonte finito. Esa igualdad es exacta para Poisson. Por ello Streamlit etiqueta λT como **referencia de tasa** bajo Polar-lognormal.

4.4 Acumulación

$$t_0 = 0, \quad t_i = t_{i-1} + \Delta_i. \quad (6)$$

El calendario conserva $t_i \leq T$. Si el jugador cae antes, otra tabla registra solo los eventos efectivamente ocurridos hasta la derrota.

5 Método de simulación

Cuadro 2: Clasificación de variables.

Categoría	Variables
Entradas	λ , T , modelo, CV, HP, DPS, alcance, radios, velocidad, dt y semilla.
Estado	Tiempo, HP, lista activa, posiciones y estado de cada enemigo.
Salidas	Supervivencia, tiempo, llegadas, bajas, remanentes, HP final y pico.

5.1 Algoritmo por paso

1. Validar dominio, geometría y precisión.
2. Separar semillas de llegadas y atributos.
3. Construir calendario y entidades reproducibles.
4. Mientras $t < T$ y $HP > 0$, definir $h = \min(dt, T - t)$.
5. Activar eventos con $t_i \leq t$ y calcular posiciones.
6. Elegir el objetivo más cercano y aplicar ataque durante h .
7. Sumar DPS de atacantes vivos y aplicar daño durante h .
8. Avanzar a $t + h$, registrar telemetría y comprobar el final.

La cota h impide integrar después de T . Una prueba coloca un enemigo en contacto exactamente al cierre y confirma que ya no puede causar daño posterior.

5.2 Parámetros base

Cuadro 3: Escenario base reproducible.

Parámetro	Valor	Unidad	Parámetro	Valor	Unidad
Horizonte	90	s	Tasa	39	llegadas/min
CV Polar	0.45	—	HP protagonista	110	HP
DPS protagonista	42	HP/s	Alcance	10	m
HP infectado	42	HP	Velocidad	1.55	m/s
DPS infectado	9	HP/s	Paso dt	0.05	s
Semilla	22193	—	Radio arena	18	m

6 Diseño experimental

6.1 Validación formal

Las muestras de interarribos tienen tamaño fijo y no sufren censura por horizonte. Se adopta $\alpha = 0.05$ y la auditoría recorre los tres niveles en que se construye una variable aleatoria.

Nivel 1, la fuente uniforme. Tanto la transformada inversa como Marsaglia Polar son funciones de uniformes: un defecto en la entrada contamina toda la salida. Se aplican cinco contrastes a PCG64 y los mismos cinco a un generador congruencial lineal propio, $x_{n+1} = (16807 x_n) \bmod (2^{31} - 1)$:

- ji-cuadrada de uniformidad sobre veinte celdas equiprobables;
- KS contra $U(0, 1)$;
- rachas ascendentes y descendentes;
- Ljung-Box conjunto para los rezagos 1 a 5;
- ji-cuadrada serial sobre tercias no solapadas del cubo unitario.

Nivel 2, las transformaciones. KS contra Exponencial(λ), KS de las normales Marsaglia contra $N(0, 1)$, Pearson de rezago uno en ambos flujos, prueba binomial de aceptación contra $\pi/4$ y contraste de la media lognormal frente a $1/\lambda$.

Nivel 3, el conteo. Índice de dispersión y ji-cuadrada de bondad de ajuste contra la función de masa Poisson(λT), ambos sobre 600 calendarios completos.

Dos decisiones merecen justificación explícita.

Por qué no se contrasta la lognormal. El estadístico de Kolmogorov-Smirnov depende solo de los valores $F(x_i)$ y es por tanto invariante ante transformaciones monótonas. Como $\Delta = e^{\mu + \sigma Z}$ es monótona en Z , un KS de Δ contra la lognormal devuelve exactamente el mismo estadístico y el mismo valor p que el KS de Z contra $N(0, 1)$: no es un contraste adicional, es el mismo número escrito dos veces. Lo que el KS de Z no puede detectar es un error en la parametrización, porque μ y σ no intervienen en él; por eso se contrasta aparte que $E[\Delta] = 1/\lambda$.

Por qué se corrige por multiplicidad. Con 18 contrastes simultáneos al 5 %, la probabilidad de que al menos uno se rechace por azar supera el 60 %. La decisión se toma sobre el valor p ajustado

por Holm-Bonferroni, que controla la tasa de error por familia sin suponer independencia entre las pruebas.

No rechazar H_0 no demuestra perfección; indica que la muestra no contradice el modelo bajo el contraste utilizado.

6.2 Monte Carlo y Wilson

Para R partidas:

$$\hat{p} = \frac{1}{R} \sum_{r=1}^R I_r, \quad (7)$$

donde $I_r = 1$ si la misión termina con vida. Se usa Wilson al 95 % por su comportamiento cerca de cero y uno [4].

6.3 Inferencia pareada

Cada corrida usa semillas emparejadas. Supervivencia se contrasta con McNemar exacta y las métricas continuas con Wilcoxon pareada. Los IC del efecto usan 4000 remuestras bootstrap de pares [5]. La dirección es:

$$\text{efecto} = \text{Poisson} - \text{Polar}. \quad (8)$$

6.4 Criterios de elección

La comparación no define **mejor** como mayor supervivencia. Evalúa ajuste, independencia, costo, claridad de supuestos, extremos y efecto sobre las salidas.

7 Resultados reproducibles

Los resultados fueron generados el 2026-09-01 mediante el script `report/generate_report_assets.py`. Las tablas completas están en `report/data`.

7.1 Calidad de generadores

Cuadro 4: Contrastes representativos de cada nivel, $\alpha = 0.05$.

Generador	Contraste	Valor p
Fuente uniforme PCG64	KS contra $U(0, 1)$	0.3006
Fuente uniforme PCG64	Serial de tercias	0.4918
LCG propio (MINSTD)	KS contra $U(0, 1)$	0.8062
LCG propio (MINSTD)	Serial de tercias	0.6986
Poisson-exponencial	KS exponencial	0.1863
Marsaglia Polar	KS normal de Z	0.9236
Marsaglia Polar	Aceptación contra $\pi/4$	0.1716
Polar-lognormal	Media contra $1/\lambda$	0.7826
Conteo Poisson	Índice de dispersión	0.5439
Conteo Poisson	Ji-cuadrada contra pmf	0.3527

Ninguno de los 18 contrastes fue rechazado. El menor valor p nominal de toda la familia fue 0.1716, de modo que la conclusión no depende de la corrección por multiplicidad: se sostiene igual antes y después de aplicarla. La tasa de aceptación observada del método polar fue 0.7765, compatible con el valor teórico $\pi/4 \approx 0.7854$. La Figura 1 inspecciona forma, cuantiles, conteos y estructura de la fuente uniforme.

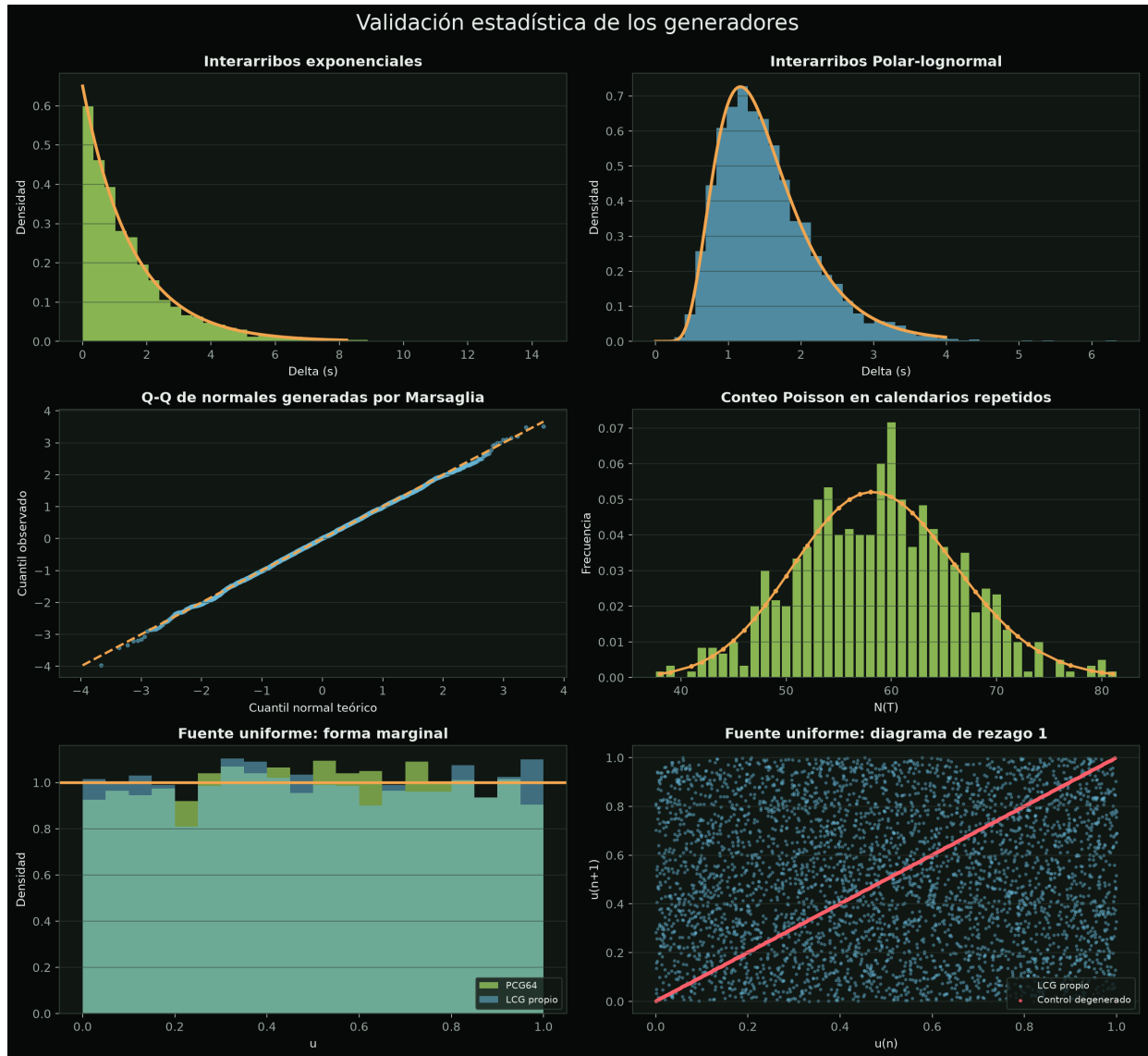


Figura 1: Histogramas, densidades, Q-Q normal, conteos Poisson y auditoría de la fuente uniforme. El diagrama de rezago uno, abajo a la derecha, contrapone el LCG propio con el control degenerado.

7.2 Control negativo

Una batería que solo aprueba generadores correctos no demuestra nada mientras no se compruebe que también reprueba a los defectuosos. Se somete a los mismos contrastes un contador puro $x_{n+1} = (x_n + 1) \bmod m$, que recorre su período completo y es por construcción perfectamente uniforme, pero también perfectamente predecible. El resultado separa dos propiedades que suelen confundirse:

Cuadro 5: Control negativo: uniforme por construcción, jamás aleatorio.

Contraste	Propiedad evaluada	Valor p
Ji-cuadrada de uniformidad	Forma marginal	0.9999
KS contra $U(0, 1)$	Forma marginal	1.0000
Rachas arriba/abajo	Orden secuencial	$< 10^{-16}$
Ljung-Box rezagos 1–5	Correlación conjunta	$< 10^{-16}$
Serial de tercias	Estructura en el cubo	$< 10^{-16}$

El contador aprueba los dos contrastes de forma con valores p cercanos a uno y fracasa en los tres de dependencia. Esta asimetría es la justificación cuantitativa de por qué la validación no se agota en un histograma: uniformidad y aleatoriedad son propiedades distintas y exigen pruebas distintas.

7.3 Costo de generación

La mediana fue 0.009 microsegundos por interarribo exponencial y 0.948 microsegundos por valor Polar-lognormal. El benchmark depende del equipo, pero evidencia el costo del rechazo y las transformaciones de Polar.

7.4 Supervivencia y congestión

Cuadro 6: Escenario base, 400 partidas por modelo.

Modelo	$\hat{p}$ (%)	IC Wilson (%)	Tiempo (s)	Pico
Poisson	96.5	[94.2, 97.9]	88.528	9.482
Polar-lognormal	100.0	[99.0, 100.0]	90.000	6.482

Cuadro 7: Contingencia pareada.

Resultado	Corridas
Ambos sobreviven	386
Solo Poisson	0
Solo Polar	14
Ambos fallan	0

La diferencia fue -3.5 puntos porcentuales, con IC [-5.5, -1.8]. McNemar produjo $p = 0.000122$: existe evidencia de diferencia en este escenario. La concurrencia fue 3.000 enemigos mayor bajo Poisson, con $p = 2.003 \times 10^{-62}$.

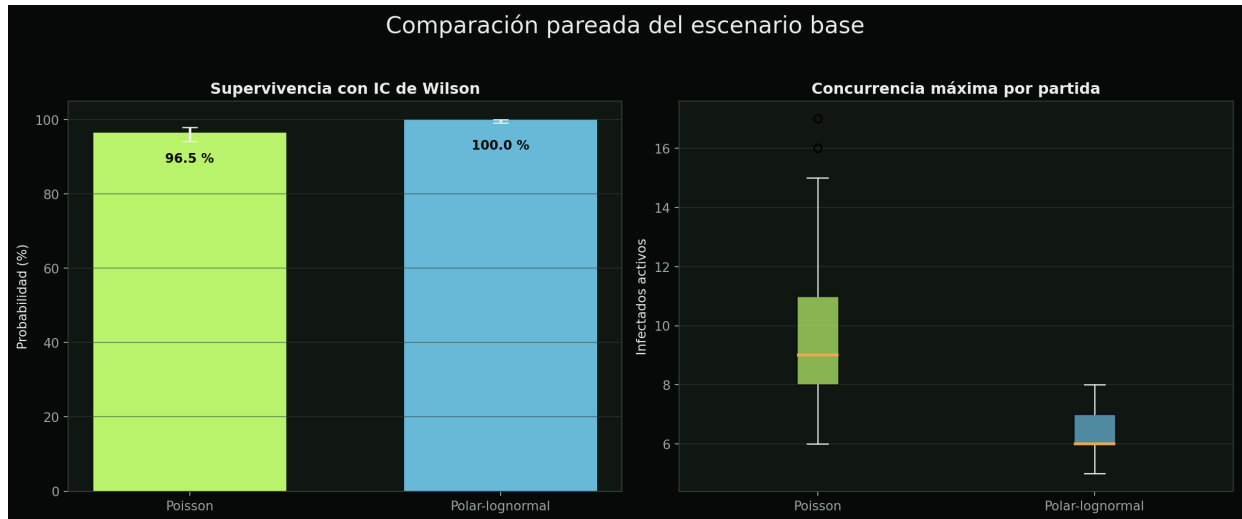


Figura 2: Supervivencia con Wilson y concurrencia máxima.

7.5 Sensibilidad a la tasa

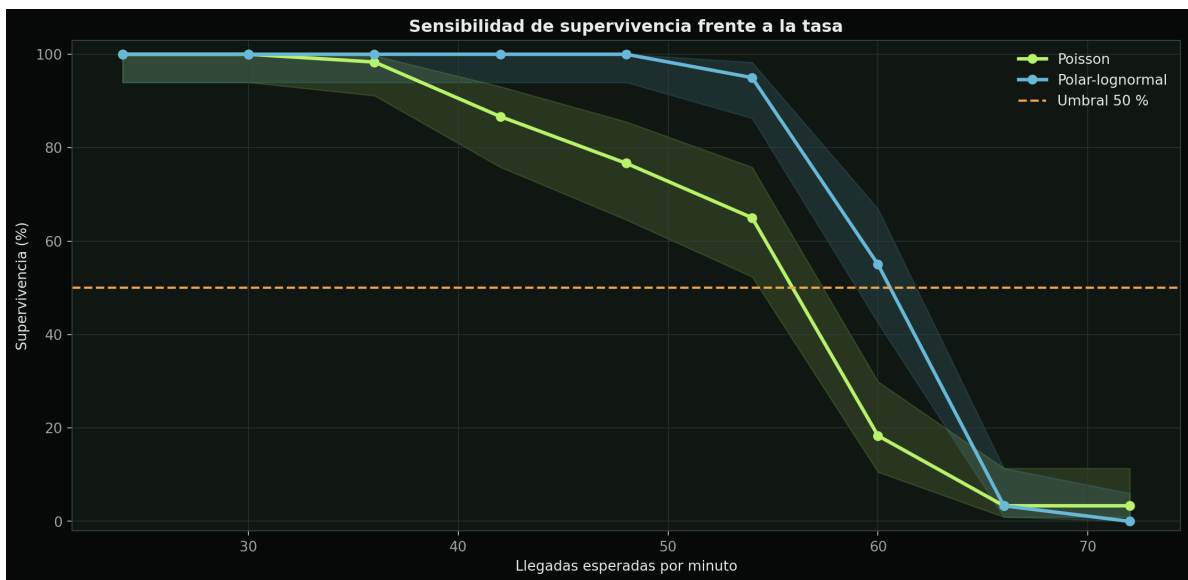


Figura 3: Supervivencia para tasas de 24 a 72 llegadas/min.

Una sola tasa no produce una conclusión universal. Al aumentar λ , la presión supera la capacidad ofensiva y cae la supervivencia. Con CV 0.45, Polar-lognormal retrasa rachas, pero esa ventaja pertenece al escenario.

7.6 Trayectoria individual



Figura 4: HP y enemigos activos con semilla 22193.

La trayectoria explica causalidad entre rachas, concurrencia y daño; la inferencia final se apoya en cientos de partidas.

8 Análisis y elección

Cuadro 8: Matriz razonada de decisión.

Criterio	Poisson-exponencial	Polar-lognormal
Fundamento	Proceso de conteo exacto	Proceso de renovación
Ajuste	No rechazado	Z normal y media calibrada
Interpretación	Un parámetro; relación exacta	Media y CV controlables
Costo	0.009 μ s/valor	0.948 μ s/valor
Rachas	CV 1; mayor concurrencia	CV 0.45; mayor regularidad
Uso	Independencia y tasa constante	Datos con regularidad positiva

Poisson permanece como modelo principal por correspondencia con el conteo, parsimonia, costo y supuestos. Polar-lognormal demuestra que igualar la media no iguala congestión y resulta útil si datos observados exigen $CV \neq 1$. Sin telemetría real, la elección está condicionada por supuestos y no constituye una verdad empírica sobre apocalipsis.

9 Visualización y arquitectura

Streamlit administra formulario, estado y caché; el motor numérico no depende de Streamlit ni Plotly. Mesh3d construye una escena navegable con personajes, ruinas, vehículos, barricadas y anillos tácticos.



Figura 5: Aparición, alcance, contacto y presión en la arena.

Pygame no es requisito de la rúbrica y se orienta principalmente a superficies 2D. Un 3D de escritorio exigiría OpenGL u otro motor. Plotly conserva controles, escena y evidencia en una aplicación web [6]. Panda3D es trabajo futuro, no una deuda de esta entrega.

10 Verificación

La suite contiene veintiuna pruebas:

1. reproducibilidad Poisson;
2. media y varianza del conteo;
3. media y positividad lognormal;
4. los 18 contrastes de referencia bajo corrección de Holm;
5. ausencia de estadísticos duplicados entre contrastes;
6. separación del control negativo respecto de la tabla principal;
7. reproducibilidad y rango del LCG propio;
8. aceptación de la batería sobre la fuente de referencia;
9. detección de dependencia en un generador uniforme pero predecible;
10. corrección de Holm contra su cálculo manual y sus cotas;
11. diseño de propuestas fijas para la prueba de aceptación;
12. potencia de la ji-cuadrada del conteo ante una media desplazada;
13. reproducibilidad de partida;
14. separación de eventos programados y ocurridos;
15. rechazo de geometría inválida;

16. igualdad de semillas pareadas;
17. estabilidad al reducir dt ;
18. ausencia de daño después de T ;
19. dirección y contingencia pareadas;
20. límites del intervalo de Wilson.

`streamlit.testing` verifica además carga, misión, validación, comparación y Monte Carlo sin excepciones.

11 Reproducibilidad

Desde `zombie_poisson_streamlit`:

```
python -m pip install -r requirements/requirements.txt
python -m unittest discover -s tests -v
python report/generate_report_assets.py
python -m streamlit run App/app.py
```

Las versiones están fijadas y los CSV permiten auditar números sin abrir la UI. Para recompilar:

```
cd report
pdflatex informe.tex
pdflatex informe.tex
```

12 Supuestos y limitaciones

1. La tasa es constante dentro de cada partida.
2. Independencia es un supuesto de diseño, no una conclusión de datos.
3. El protagonista es estacionario y enfoca un objetivo.
4. No hay colisiones ni evitación de obstáculos.
5. El combate usa paso finito; la convergencia cubre un caso sencillo.
6. La variación uniforme de atributos no tiene calibración empírica.
7. KS depende del tamaño; se complementa con momentos y Q-Q.
8. El LCG propio se audita, pero no sustituye a PCG64 dentro del motor: su papel es servir de término de comparación con evidencia.
9. El control negativo demuestra potencia frente a dependencia secuencial; no cubre todas las formas posibles de defecto.
10. Valores p no miden importancia práctica; se reportan efectos e IC.
11. El benchmark depende del equipo.
12. Las curvas Monte Carlo usan una cuadrícula finita.
13. Las ruinas son visuales y no alteran trayectorias.

13 Conclusiones

1. Los 18 contrastes fueron coherentes con los generadores declarados, incluida la fuente uniforme que ambos métodos comparten, y ninguno se solapa con otro.
2. El control negativo confirma que la batería distingue uniformidad de aleatoriedad: un contador aprueba los contrastes de forma y fracasa en los de dependencia.
3. Igual media no produjo igual concurrencia: Poisson generó un pico medio aproximadamente tres enemigos mayor.
4. El diseño pareado encontró diferencia de supervivencia en el escenario base sin convertirla en superioridad universal.
5. Poisson es principal por coherencia, parsimonia, costo e interpretación.
6. Polar-lognormal es valioso cuando interesa controlar el CV.
7. La corrección temporal impide daño después del horizonte.
8. App, CSV, figuras, informe y presentación usan evidencia reproducible.

14 Trabajo futuro

Se propone calibrar λ y atributos con telemetría real, evaluar Poisson no homogéneo $\lambda(t)$, estudiar conjuntamente CV y combate, permitir movimiento, hacer funcionales los obstáculos y crear una versión Panda3D. Estas extensiones no son necesarias para cumplir la rúbrica actual.

Referencias

- [1] S. M. Ross, **Simulation**, Academic Press.
- [2] L. Devroye, **Non-Uniform Random Variate Generation**, Springer, 1986.
- [3] G. Marsaglia y T. A. Bray, **A convenient method for generating normal variables**, SIAM Review, 1964.
- [4] E. B. Wilson, **Probable inference, the law of succession, and statistical inference**, JASA, 1927.
- [5] B. Efron y R. Tibshirani, **An Introduction to the Bootstrap**, Chapman & Hall/CRC, 1993.
- [6] Plotly, **3D Mesh Plots in Python**.
<https://plotly.com/python/3d-mesh/>